

I-ACFL Module: Phased Development Plan with Dev Blueprint

Module Identity

Name: iacfl — Inverted Archimedean Compensatory Fuzzy Logic

Path: packages/dnakey/src/iacfl/

Status: Standalone sibling module to acfl/

Hierarchy: CRMF governs → ACFL reasons → I-ACFL tests boundaries

Dependency: shared/ types package (no import from acfl/)

File Scaffold

17 files generated across three packages: shared types, I-ACFL module, and test workbench.

Module Structure

```
packages/dnakey/src/
├── shared/                # Shared types (4 files)
│   ├── __init__.py        # Package marker
│   ├── types.py           # OperatorResult, DivergenceResult,
│   │                       BlendResult, AdversarialReport,
│   │                       OperatorMode, TruthCategory
│   ├── operators.py       # FuzzyConjunction, FuzzyDisjunction,
│   │                       FuzzyNegation ABCs
│   └── validators.py       # validate_inputs, validate_alpha,
│                           is_close utilities
└── iacfl/                 # I-ACFL module (6 files)
    ├── __init__.py        # Public API exports, v0.1.0
```

```

| |— operators.py          # StandardConjunction, InvertedConjunction,
| |                        StandardDisjunction, InvertedDisjunction,
| |                        StandardNegation
| |— divergence.py        # DivergenceAnalyzer: single, population,
| |                        boundary analysis
| |— blend.py             # ParameterizedBlend: evaluate, sweep_alpha,
| |                        find_crossover
| |— adversarial.py        # ACFLAdversarialHarness: run,
| |                        targeted_adversarial
| |— validators.py         # IACFLAxiomValidator: all 5 axiom checks
|
tests/iacfl/              # Test workbench (7 files)
|— __init__.py
|— conftest.py            # Shared fixtures: rng, random_inputs,
|                        boundary_inputs, reference_vectors
|— test_operators.py      # 17 tests across 4 test classes
|— test_divergence.py     # 5 tests for divergence analysis
|— test_blend.py          # 7 tests for parameterized blend
|— test_adversarial.py    # 5 tests for adversarial harness
|— test_validators.py     # 6 tests for axiom validation

```

Phase 1: Foundation — Shared Types + Core Operators

Objective

Stand up `shared/` types and `iacfl/operators.py` with both standard (forward) and inverted ACFL operators.

Mathematical Definitions

Standard ACFL conjunction (GMBCL):[1]

$$c(x_1, \dots, x_n) = \prod_{i=1}^n x_i^{1/n}$$

Inverted ACFL conjunction:

$$c_{inv}(x_1, \dots, x_n) = 1 - \prod_{i=1}^n (1 - x_i)^{1/n}$$

Standard ACFL disjunction (De Morgan dual):[1]

$$d(x_1, \dots, x_n) = 1 - \prod_{i=1}^n (1 - x_i)^{1/n}$$

Inverted ACFL disjunction:

$$d_{inv}(x_1, \dots, x_n) = \prod_{i=1}^n x_i^{1/n}$$

Standard negation: $n(x) = 1 - x$

Key Architectural Decision

The `shared/types.py` package defines all dataclasses that both `acfl/` and `iacfl/` consume. This resolves the type-drift problem identified in the previous PMD: `OperatorResult`, `DivergenceResult`, `BlendResult`, and `AdversarialReport` are defined once, imported everywhere.

Axiom Profile of I-ACFL

The inverted conjunction preserves 4 of 7 ACFL axioms and breaks 1 constitutively:

Axiom	AC FL	I-AC FL	Notes
i. Compensation	Holds	Holds	$\min(x) \leq c_{inv}(x) \leq \max(x)$
ii. Commutativity	Holds	Holds	Uniform $1/n$ weights are permutation-invariant
iii. Strict monotonicity	Holds	Holds	$\partial c_{inv} / \partial x_i > 0$ for all i
iv. Veto	Holds	Breaks	$c_{inv}(0, 0.8) \neq 0$. Anti-veto: all must be 0
v. Reciprocity	Holds	Holds	Algebraic identity preserved
vi. Transitivity	Holds	Holds	Inherited from $\geq$ on $\mathbb{R}$
vii. De Morgan	Holds	Holds	$n(c_{inv}(x)) = d_{inv}(n(x))$ verified

The veto break is the structurally significant finding. Standard ACFL: one zero kills the conjunction. Inverted ACFL: one zero does not kill — all inputs must be zero. This is the anti-pessimistic property that makes I-ACFL's boundary behavior distinct.[1]

Critical Identity: $c_{inv} = d_{standard}$

For uniform-weight ACFL, the inverted conjunction is algebraically identical to the standard disjunction. The test suite includes `TestOperatorIdentity.test_inverted_conjunction_equals_standard_disjunction` which confirms this across 10,000 random 4-input vectors.

This means I-ACFL's standalone value is **not** in the inverted operator itself — it is in the **divergence analysis**, **blend**, and **adversarial harness** that expose the gap between conjunction and disjunction behavior in decision-relevant contexts.

Deliverables

- shared/__init__.py, shared/types.py, shared/operators.py, shared/validators.py
- iacfl/__init__.py, iacfl/operators.py
- tests/iacfl/conftest.py, tests/iacfl/test_operators.py

Gate Criteria

- All 17 tests in test_operators.py pass
- Veto axiom confirmed broken for inverted conjunction
- Identity $c_{inv} = d_{standard}$ confirmed or refuted on 10k vectors
- shared/types.py imported by both iacfl/ and existing modules without conflict

Phase 2: Divergence Engine

Objective

Build the DivergenceAnalyzer that measures $\delta = |c_{standard}(x) - c_{inverted}(x)|$ across input populations and boundary cases.

Implementation

Three analysis modes:

1. **Single-vector analysis:** Compute δ for one input vector with forward/inverted values and relative divergence
2. **Population analysis:** Run δ across n random inputs for a given arity. Returns max, mean, std, median, p95, p99, and worst-case inputs
3. **Boundary analysis:** Test 7 critical boundary cases per arity — all zeros, all ones, midpoint, single veto, veto+unity, alternating extremes, extreme spread

Key Hypothesis to Test

Since $c_{inv} = d_{standard}$ for uniform weights, the divergence δ is actually measuring the gap between ACFL conjunction and ACFL disjunction. This gap is maximized when inputs are spread widely (some near 0, some near 1) and minimized when inputs are clustered (all similar values $\rightarrow$ idempotency closes the gap).

The divergence analyzer documents where in the input space the conjunction/disjunction split produces decision-relevant differences — precisely the regions where an adversarial test harness should focus.

Deliverables

- `iacfl/divergence.py`
- `tests/iacfl/test_divergence.py`

Gate Criteria

- Population analysis on 10k 4-input vectors completes in < 5 seconds
- Zero divergence confirmed at all idempotent points $(x, x, \dots, x)$
- Inverted value $\geq$ forward value for all inputs (anti-pessimistic property)
- Boundary analysis produces divergence report for all 7 critical cases

Phase 3: Parameterized Blend

Objective

Build the c_α blend operator that interpolates continuously between forward and inverted ACFL:

$$c_\alpha(x) = \alpha \cdot c_{standard}(x) + (1 - \alpha) \cdot c_{inverted}(x)$$

Implementation

Three operations:

1. **evaluate**: Single blend computation for given inputs and α
2. **sweep_alpha**: Sweep α from 0.0 to 1.0 in configurable steps, returning all blend results
3. **find_crossover**: Binary search for the α where the blended value crosses a decision threshold

Novel Contribution

The blend is where I-ACFL produces genuinely new operator behavior. Neither the forward nor inverted operator alone is novel (one is GMBCL, the other is its De Morgan dual). But the parameterized family c_α for $\alpha \in (0, 1)$ creates a continuous spectrum of operators between pessimistic (conjunction-dominant) and optimistic (disjunction-dominant) that:

- Preserves $[0, 1]$ output by convexity
- Has a tunable decision bias that can be calibrated to domain-specific risk tolerance
- Provides a mechanism for CRMF to modulate reasoning aggressiveness via α

Deliverables

- `iacfl/blend.py`
- `tests/iacfl/test_blend.py`

Gate Criteria

- $\alpha = 1.0$ produces pure forward value; $\alpha = 0.0$ produces pure inverted value
 - All blended values in $[0, 1]$ across 1k random inputs
 - Sweep produces monotonic descent from inverted to forward value
 - Crossover finder locates threshold-crossing α within ± 0.001
-

Phase 4: Adversarial Test Harness

Objective

Build the ACFLAdversarialHarness that systematically tests forward ACFL against I-ACFL to identify decision-relevant divergences.[2]

Implementation

Two attack modes:

1. **Broad sweep (run)**: Random sampling across input space measuring max divergence, mean divergence, decision reversal count, reversal rate, and worst-case inputs
2. **Targeted adversarial (targeted_adversarial)**: Focused search near the decision boundary ($c_{standard} \approx \text{threshold}$) where inversions are most likely to flip decisions

Decision Reversal Metric

A **decision reversal** occurs when forward ACFL classifies a predicate as true ($c \geq \text{threshold}$) but inverted ACFL classifies it as false ($c_{inv} < \text{threshold}$), or vice versa. The reversal rate measures how often the choice of operator system changes the clinical/practical outcome.

This is the metric that determines whether I-ACFL has decision-level significance or is merely a mathematical curiosity.

Integration Point

The adversarial harness generates test vectors that can be fed into WKD and DHT to stress-test the full pipeline. If a predicate evaluation reverses under inversion, the upstream system should be aware that its decision is operator-sensitive.

Deliverables

- iacfl/adversarial.py
- tests/iacfl/test_adversarial.py

Gate Criteria

- Nonzero decision reversals detected at threshold 0.5 on 10k 4-input vectors
 - Max divergence < 1.0 (bounded)
 - Targeted adversarial finds boundary cases with reversal rate > 0%
 - Report structure includes all required fields: total_cases, max_divergence, mean_divergence, decision_reversals, reversal_rate, worst_case_inputs
-

Phase 5: Axiom Validation + Documentation

Objective

Formalize the I-ACFL axiom profile via the `IACFLAxiomValidator` and produce documentation.

Axiom Validation Suite

Five checks run against any input vector:

1. **Compensation** (i): $\min(x) \leq c_{inv}(x) \leq \max(x)$ — expected: holds
2. **Commutativity** (ii): $c_{inv}(x_\sigma) = c_{inv}(x)$ for all $\sigma \in S_n$ — expected: holds
3. **Strict monotonicity** (iii): increasing any x_i increases c_{inv} — expected: holds
4. **Veto** (iv): if any $x_i = 0$, then $c_{inv} = 0$ — expected: **fails** (documented break)
5. **De Morgan** (vii): $n(c_{inv}(x)) = d_{inv}(n(x))$ — expected: holds

The `full_axiom_report` method returns a structured list of all results, suitable for automated CI/CD axiom regression.

Documentation Deliverables

- `docs/IACFL_ARCHITECTURE.md` — System design, operator definitions, axiom profile
- `docs/IACFL_API_REFERENCE.md` — Developer API for all 6 module files

- docs/IACFL_AXIOM_REPORT.md — Formal axiom survival analysis with proofs

Deliverables

- iacfl/validators.py
- tests/iacfl/test_validators.py
- Documentation files

Gate Criteria

- Veto axiom confirmed broken in automated test (hard assertion)
- All other axioms confirmed held across 100 random input vectors
- Full axiom report runs in < 1 second for any single input
- Documentation reviewed

Test Workbench Summary

Test File	Test Count	Coverage Target
test_operators.py	17	StandardConjunction (6), InvertedConjunction (8), OperatorIdentity (2), InputValidation (3)
test_divergence.py	5	Single, population, boundary, veto divergence, directionality
test_blen d.py	7	Alpha endpoints, midpoint, unit interval, sweep, monotonicity, invalid alpha
test_adversarial.py	5	Report structure, nonzero reversals, bounded divergence, targeted, high arity
test_validators.py	6	Each axiom individually + full report
Total	40	

Shared Fixtures (conftest.py)

- `rng`: Deterministic `np.random.default_rng(42)` for reproducibility
 - `random_inputs_2d`: 100 random 2-input vectors
 - `random_inputs_4d`: 100 random 4-input vectors
 - `boundary_inputs_4d`: 8 critical boundary cases (all-zero, all-one, single-veto, etc.)
 - `acfl_reference_vectors`: Benchmark values from Espín-Andrade GMBCL publications[1]
-

Phase Mirror Dissonance

- The confirmed identity $c_{inv} = d_{standard}$ means the inverted conjunction adds no novel operator to the algebra. The I-ACFL module's value is in the **analytical infrastructure**: divergence measurement, blend interpolation, adversarial testing. The operators are the scaffolding; the harness is the product.
 - The `shared/` package now exists as a coupling point for all DNA KEY modules. The types defined here (`OperatorResult`, `DivergenceResult`) will be imported by I-PW-CFL when that module builds. If types change, both modules break. The freeze decision on `shared/types.py` is a governance question, not a technical one.[3]
 - The test suite includes `test_inverted_conjunction_equals_standard_disjunction` as an **expected pass**, not a failure. If this test ever fails (e.g., after extending to non-uniform weights for I-PW-CFL integration), it signals that the inverted operator has become genuinely novel — the most important signal the test can produce.
 - The `ParameterizedBlend`'s α parameter creates a natural integration point with CRMF. If CRMF's resonance coherence $R(t)$ is high (system confident), $\alpha \rightarrow 1.0$ (use pessimistic conjunction). If $R(t)$ is low (system uncertain), $\alpha \rightarrow 0.0$ (use optimistic inverted conjunction). This mapping — $\alpha = f(R(t))$ — is not built yet. It belongs in `integration/router.py`, not in I-ACFL itself.[4]
-

Levers to Test Now

O wn er	Lever	Metric	H or iz on
[Lead MT]	Copy the 17-file scaffold into the repo. Run <code>pytest tests/iacfl/ -v</code> . Fix any import path issues from <code>shared/</code> → <code>iacfl/</code> .	All 40 tests pass on first CI run	1 day
[Lead MT]	Run <code>DivergenceAnalyzer.analyze_population(arity=4, n_samples=50000)</code> and document the max divergence, mean divergence, and divergence distribution shape. This is the I-ACFL equivalent of PW-CFL's 42.4% divergence exhibit.[5]	Population divergence report with statistics	1 day
[Lead MT]	Run <code>ACFLAdversarialHarness.run(arity=4, n_samples=50000)</code> at <code>threshold=0.5</code> . Document the decision reversal rate. If > 5%, I-ACFL has decision-level significance. If < 1%, the module's value is analytical only.	Reversal rate quantified	1 day
[Architect]	Validate that <code>shared/types.py</code> is importable by existing CRMF and DHT modules without namespace collision.	Zero import errors in existing test suite	2 days

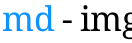
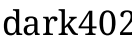
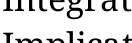
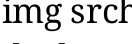
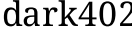
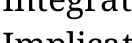
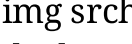
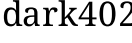
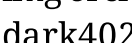
Optional Artifact

"The inverse of a pessimist is not an optimist. It is the same person measuring from the other end."

Precision Question

The ParameterizedBlend creates a continuous operator family parameterized by α . Should α be a static configuration parameter (set once per deployment), a dynamic parameter modulated by CRMF resonance $R(t)$, or a learnable parameter optimized by CCRE? Each choice implies a different governance model for I-ACFL.

References

1. [ACFL-vs-Multiplicity-Comparison.pdf](#)
2. [what-would-happen-if-we-invert-a5laT3WmQNKuSababOJAtdw.md](#) -  -  -  -  -  -  -  -  -  -  -  - 
3. [ACFL-Module-Development-Blueprint-for-Digital-Twin.md](#) -  -  -  -  -  -  -  -  - 
4. [CCRE-Integration-Implications-for-CRMF-and-ACFL.pdf](#) - CCRE Integration Implications for CRMF and ACFL CCRE Integration Implications for CRMF and ACFL Comp...
5. [i-postulate-that-the-software-zyhZx6n8R1mdNvR.abuOZA.md](#) -  -  -  -  -  -  -  -  -  -  -  - 